2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Gemini 3.6 Flash & Claude Sonnet 5.5

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both GenAI programs through their websites (claude.ai & gemini.google.com respectively)

c. Prompt used. (You must use the same prompt for both GenAIs.)

Generate C++ code using objects, not functions, to implement the following requirements:

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:
Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails
• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).
• Sample Test File:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
• Your program should display this:
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.

d. Generated code from both GenAIs.
GEMINI'S GENERATION:

```
#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
```

```
// Class representing an Email object
class Email
{
private:
   std::string senderCategory;
   std::string subject;
   std::string date; // MM-DD-YYYY

   // Helper to assign numeric weight to categories
   int getCategoryPriority() const
   {
       if (senderCategory == "Boss")
           return 5;
       if (senderCategory == "Subordinate")
           return 4;
       if (senderCategory == "Peer")
           return 3;
       if (senderCategory == "ImportantPerson")
           return 2;
       if (senderCategory == "OtherPerson")
           return 1;
       return 0;
   }

   // Helper to convert MM-DD-YYYY into YYYYMMDD string for simple chronological
comparison
   std::string getFormattedDate() const
   {
       if (date.length() == 10)
       {
           std::string month = date.substr(0, 2);
           std::string day = date.substr(3, 2);
           std::string year = date.substr(6, 4);
           return year + month + day;
       }
       return date;
   }

public:
   Email() : senderCategory(""), subject(""), date("") {}

   Email(const std::string &sender, const std::string &subj, const std::string &d)
```

```
        : senderCategory(sender), subject(subj), date(d) {}

    std::string getSenderCategory() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }

    // Custom comparison operators based on priority hierarchy & newest date
    bool operator>(const Email &other) const
    {
        if (getCategoryPriority() != other.getCategoryPriority())
        {
            return getCategoryPriority() > other.getCategoryPriority();
        }
        return getFormattedDate() > other.getFormattedDate();
    }

    bool operator<(const Email &other) const
    {
        if (getCategoryPriority() != other.getCategoryPriority())
        {
            return getCategoryPriority() < other.getCategoryPriority();
        }
        return getFormattedDate() < other.getFormattedDate();
    }
};

// Node structure for pointer/list-based tree nodes
struct Node
{
    Email data;
    Node *left;
    Node *right;
    Node *parent;

    Node(const Email &email)
        : data(email), left(nullptr), right(nullptr), parent(nullptr) {}
};

// Dynamic linked queue used strictly for Level-Order tree traversal
class NodeQueue
{
private:
```

```
    struct QueueNode
    {
        Node *treeNode;
        QueueNode *next;
        QueueNode(Node *node) : treeNode(node), next(nullptr) {}
    };

    QueueNode *head;
    QueueNode *tail;

public:
    NodeQueue() : head(nullptr), tail(nullptr) {}

    ~NodeQueue()
    {
        while (!isEmpty())
        {
            dequeue();
        }
    }

    bool isEmpty() const
    {
        return head == nullptr;
    }

    void enqueue(Node *node)
    {
        QueueNode *newNode = new QueueNode(node);
        if (tail == nullptr)
        {
            head = tail = newNode;
        }
        else
        {
            tail->next = newNode;
            tail = newNode;
        }
    }

    Node *dequeue()
    {
```

```
        if (isEmpty())
            return nullptr;
        QueueNode *temp = head;
        Node *node = temp->treeNode;
        head = head->next;
        if (head == nullptr)
        {
            tail = nullptr;
        }
        delete temp;
        return node;
    }
};

// Pointer-based (list-based) MaxHeap implementation
class MaxHeap
{
private:
    Node *root;
    int count;

    void swapData(Node *a, Node *b)
    {
        Email temp = a->data;
        a->data = b->data;
        b->data = temp;
    }

    void heapifyUp(Node *node)
    {
        while (node->parent != nullptr && node->data > node->parent->data)
        {
            swapData(node, node->parent);
            node = node->parent;
        }
    }

    void heapifyDown(Node *node)
    {
        while (node != nullptr)
        {
            Node *largest = node;
```

```
        if (node->left != nullptr && node->left->data > largest->data)
        {
            largest = node->left;
        }
        if (node->right != nullptr && node->right->data > largest->data)
        {
            largest = node->right;
        }

        if (largest != node)
        {
            swapData(node, largest);
            node = largest;
        }
        else
        {
            break;
        }
    }
}

// Level-order helper to locate insertion parent node
Node *getInsertionParent()
{
    if (root == nullptr)
        return nullptr;

    NodeQueue q;
    q.enqueue(root);

    while (!q.isEmpty())
    {
        Node *curr = q.dequeue();
        if (curr->left == nullptr || curr->right == nullptr)
        {
            return curr;
        }
        q.enqueue(curr->left);
        q.enqueue(curr->right);
    }
    return nullptr;
```

```
    }

    // Level-order helper to find the last node in complete binary tree
    Node *getLastNode()
    {
        if (root == nullptr)
            return nullptr;

        NodeQueue q;
        q.enqueue(root);
        Node *last = nullptr;

        while (!q.isEmpty())
        {
            last = q.dequeue();
            if (last->left)
                q.enqueue(last->left);
            if (last->right)
                q.enqueue(last->right);
        }
        return last;
    }

    void clearTree(Node *node)
    {
        if (node != nullptr)
        {
            clearTree(node->left);
            clearTree(node->right);
            delete node;
        }
    }

public:
    MaxHeap() : root(nullptr), count(0) {}

    ~MaxHeap()
    {
        clearTree(root);
    }

    int size() const
```

```
{
    return count;
}

bool isEmpty() const
{
    return count == 0;
}

void insert(const Email &email)
{
    Node *newNode = new Node(email);
    count++;

    if (root == nullptr)
    {
        root = newNode;
        return;
    }

    Node *parent = getInsertionParent();
    newNode->parent = parent;

    if (parent->left == nullptr)
    {
        parent->left = newNode;
    }
    else
    {
        parent->right = newNode;
    }

    heapifyUp(newNode);
}

Email getMax() const
{
    if (isEmpty())
    {
        return Email();
    }
    return root->data;
```

```
    }

    void removeMax()
    {
        if (isEmpty())
            return;

        if (count == 1)
        {
            delete root;
            root = nullptr;
            count = 0;
            return;
        }

        Node *lastNode = getLastNode();
        root->data = lastNode->data;

        if (lastNode->parent != nullptr)
        {
            if (lastNode->parent->left == lastNode)
            {
                lastNode->parent->left = nullptr;
            }
            else
            {
                lastNode->parent->right = nullptr;
            }
        }

        delete lastNode;
        count--;

        heapifyDown(root);
    }
};

// CEO Inbox Controller Object managing command parsing
class CEOInboxManager
{
private:
    MaxHeap priorityQueue;
```

```
public:
   void processCommand(const std::string &line)
   {
       if (line.empty())
           return;

       if (line.rfind("EMAIL ", 0) == 0)
       {
           std::string payload = line.substr(6);
           std::stringstream ss(payload);
           std::string sender, subject, date;

           std::getline(ss, sender, ',');
           std::getline(ss, subject, ',');
           std::getline(ss, date, ',');

           Email newEmail(sender, subject, date);
           priorityQueue.insert(newEmail);
       }
       else if (line == "COUNT")
       {
           std::cout << "There are " << priorityQueue.size() << " emails to read." <<
std::endl
                     << std::endl;
       }
       else if (line == "NEXT")
       {
           if (!priorityQueue.isEmpty())
           {
               Email top = priorityQueue.getMax();
               std::cout << "Next email:" << std::endl;
               std::cout << "    Sender: " << top.getSenderCategory() << std::endl;
               std::cout << "    Subject: " << top.getSubject() << std::endl;
               std::cout << "    Date: " << top.getDate() << std::endl
                         << std::endl;
           }
       }
       else if (line == "READ")
       {
           if (!priorityQueue.isEmpty())
           {
```

```
                priorityQueue.removeMax();
            }
        }
    }
};

int main(int argc, char *argv[])
{
    CEOInboxManager inbox;

    // Read commands from file if passed as command line argument, else fallback to
standard input
    if (argc > 1)
    {
        std::ifstream inputFile(argv[1]);
        if (inputFile.is_open())
        {
            std::string line;
            while (std::getline(inputFile, line))
            {
                inbox.processCommand(line);
            }
            inputFile.close();
            return 0;
        }
    }

    std::string line;
    while (std::getline(std::cin, line))
    {
        inbox.processCommand(line);
    }

    return 0;
}
```

CLAUDE'S GENERATION:

```
// CEO Email Prioritizer
// Uses a hand-built array-based MaxHeap as a priority queue.
// Build: g++ -std=c++11 -o ceo_email ceo_email.cpp
// Run:   ./ceo_email testfile.txt   (or run with no args and enter the file name)
```

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// ---------------------------------------------------------------------------
// Email: holds one message and knows how to compare its priority to another.
// ---------------------------------------------------------------------------
class Email
{
private:
   string sender;
   string subject;
   string date;    // MM-DD-YYYY
   int senderRank; // higher = read sooner
   int dateKey;    // YYYYMMDD, higher = newer
   long arrival;   // tie-breaker: earlier arrival wins

   static int rankFor(const string &s)
   {
       if (s == "Boss")
           return 5;
       if (s == "Subordinate")
           return 4;
       if (s == "Peer")
           return 3;
       if (s == "ImportantPerson")
           return 2;
       return 1; // OtherPerson
   }

   static int keyFor(const string &d)
   {
       // MM-DD-YYYY -> YYYYMMDD
       if (d.size() < 10)
           return 0;
       int mm = stoi(d.substr(0, 2));
       int dd = stoi(d.substr(3, 2));
       int yyyy = stoi(d.substr(6, 4));
       return yyyy * 10000 + mm * 100 + dd;
   }
```

```
public:
   Email() : senderRank(0), dateKey(0), arrival(0) {}

   Email(const string &snd, const string &subj, const string &dt, long order)
       : sender(snd), subject(subj), date(dt),
         senderRank(rankFor(snd)), dateKey(keyFor(dt)), arrival(order) {}

   // true if this email should be read BEFORE other
   bool higherPriorityThan(const Email &other) const
   {
       if (senderRank != other.senderRank)
           return senderRank > other.senderRank;
       if (dateKey != other.dateKey)
           return dateKey > other.dateKey; // newest first
       return arrival < other.arrival;     // stable tie-break
   }

   void display() const
   {
       cout << "Next email:" << endl;
       cout << "\tSender: " << sender << endl;
       cout << "\tSubject: " << subject << endl;
       cout << "\tDate: " << date << endl;
       cout << endl;
   }
};

// ---------------------------------------------------------------------------
// MaxHeap: list-based (dynamic array) binary max-heap of Emails.
// ---------------------------------------------------------------------------
class MaxHeap
{
private:
   Email *data;
   int count;
   int capacity;

   static int parent(int i) { return (i - 1) / 2; }
   static int left(int i) { return 2 * i + 1; }
   static int right(int i) { return 2 * i + 2; }
```

```
void grow()
{
    int newCap = capacity * 2;
    Email *bigger = new Email[newCap];
    for (int i = 0; i < count; i++)
        bigger[i] = data[i];
    delete[] data;
    data = bigger;
    capacity = newCap;
}

void swapAt(int a, int b)
{
    Email tmp = data[a];
    data[a] = data[b];
    data[b] = tmp;
}

void siftUp(int i)
{
    while (i > 0 && data[i].higherPriorityThan(data[parent(i)]))
    {
        swapAt(i, parent(i));
        i = parent(i);
    }
}

void siftDown(int i)
{
    while (true)
    {
        int l = left(i), r = right(i), best = i;
        if (l < count && data[l].higherPriorityThan(data[best]))
            best = l;
        if (r < count && data[r].higherPriorityThan(data[best]))
            best = r;
        if (best == i)
            break;
        swapAt(i, best);
        i = best;
    }
}
```

```
public:
   MaxHeap() : data(new Email[16]), count(0), capacity(16) {}

   ~MaxHeap() { delete[] data; }

   // Disable copying (owns raw memory)
   MaxHeap(const MaxHeap &) = delete;
   MaxHeap &operator=(const MaxHeap &) = delete;

   void insert(const Email &e)
   {
       if (count == capacity)
           grow();
       data[count] = e;
       siftUp(count);
       count++;
   }

   bool isEmpty() const { return count == 0; }
   int size() const { return count; }

   // Caller must check isEmpty() first.
   const Email &peekMax() const { return data[0]; }

   void removeMax()
   {
       if (count == 0)
           return;
       data[0] = data[count - 1];
       count--;
       if (count > 0)
           siftDown(0);
   }
};

// ---------------------------------------------------------------------------
// Inbox: the CEO's inbox; wraps the heap and executes file commands.
// ---------------------------------------------------------------------------
class Inbox
{
private:
```

```
    MaxHeap heap;
    long nextOrder;

    static string trim(const string &s)
    {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == string::npos)
            return "";
        size_t b = s.find_last_not_of(" \t\r\n");
        return s.substr(a, b - a + 1);
    }

    void addEmail(const string &fields)
    {
        // fields: "<sender>,<subject>,<date>"
        size_t c1 = fields.find(',');
        size_t c2 = (c1 == string::npos) ? string::npos : fields.find(',', c1 + 1);
        if (c1 == string::npos || c2 == string::npos)
            return; // malformed line
        string sender = trim(fields.substr(0, c1));
        string subject = trim(fields.substr(c1 + 1, c2 - c1 - 1));
        string date = trim(fields.substr(c2 + 1));
        heap.insert(Email(sender, subject, date, nextOrder++));
    }

public:
    Inbox() : nextOrder(0) {}

    void count() const
    {
        cout << "There are " << heap.size() << " emails to read." << endl;
        cout << endl;
    }

    void next() const
    {
        if (heap.isEmpty())
        {
            cout << "No emails to read." << endl;
            cout << endl;
            return;
        }
```

```
        heap.peekMax().display();
    }

    void read()
    {
        heap.removeMax(); // no-op if empty
    }

    void processLine(const string &rawLine)
    {
        string line = trim(rawLine);
        if (line.empty())
            return;

        if (line.compare(0, 6, "EMAIL ") == 0)
        {
            addEmail(line.substr(6));
        }
        else if (line == "NEXT")
        {
            next();
        }
        else if (line == "READ")
        {
            read();
        }
        else if (line == "COUNT")
        {
            count();
        }
        // unknown lines are ignored
    }
};

// ------------------------------------------------------------------------
int main(int argc, char *argv[])
{
    string filename;
    if (argc > 1)
    {
        filename = argv[1];
    }
```

```
    else
    {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }

    ifstream in(filename);
    if (!in)
    {
        cerr << "Could not open file: " << filename << endl;
        return 1;
    }

    Inbox inbox;
    string line;
    while (getline(in, line))
    {
        inbox.processLine(line);
    }
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness: Both Claude and Gemini's code use classes and implement a MaxHeap to prioritize emails by sender category and date. Claude compares emails by category rank, date, and arrival order. Gemini compares them by category priority and date, but does not include a tie-breaker for emails with the same category and date. Gemini converts dates from MM-DD-YYYY to YYYYMMDD so they can be compared chronologically. It uses a pointer-based complete binary tree and maintains the heap with heapifyUp and heapifyDown. When the inbox is empty, Claude's NEXT prints "No emails to read," while Gemini's prints nothing. Both programs do nothing when READ is used on an empty inbox. Gemini reads commands from a file or standard input, while Claude prompts for a filename if one is not provided. Gemini indents the NEXT output with four spaces, while Claude uses a tab. Gemini does not remove extra spaces after commas or carriage returns from Windows line endings. These can cause extra spaces in stored fields or make commands such as COUNT, NEXT, and READ fail. Claude removes these spaces and line endings. Claude's date conversion uses stoi, which could crash on a ten-character date containing non-digit characters. Gemini's date conversion also does not fully validate dates. Neither original program fully validates its input.

Execution Time: Claude's array-based MaxHeap has O(logn) insertion and removal. Insertion is amortized O(logn) because the array occasionally doubles in size, which requires copying the existing emails and takes O(n) time for that insertion. Checking the next email and counting emails take O(1) time. Gemini also uses heap operations that take O(logn), but it performs a level-order traversal to find the insertion parent and last node. These searches take O(n), making insertion and removal O(n) overall. Getting the highest-priority email and counting emails take O(1). Overall, Claude's implementation has faster insertion and removal, especially with larger inboxes.

Space Complexity: Both implementations use O(n) space to store emails. Claude uses a dynamically resized array that doubles when full. After resizing, some of its capacity may be unused. Each email also stores a category rank, date key, and arrival order. Gemini stores each email in a tree node with pointers to its left child, right child, and parent. It also uses a temporary linked queue for level-order traversal, which can require O(n) space. Both have linear space complexity. Claude has no per-node pointers but may have unused array capacity, while Gemini uses three pointers per tree node and additional temporary queue space, so neither clearly uses less memory overall.

Maintainability: Claude's code is organized into three classes: Email, MaxHeap, and Inbox. Each has a clear responsibility, making the program easier to follow. Gemini uses Email, MaxHeap, NodeQueue, and CEOInboxManager to separate email information, heap management, tree traversal, and command processing. Both programs require manual memory management: Claude uses a raw array, while Gemini creates and deletes tree nodes. Claude disables copying of its MaxHeap, while Gemini's MaxHeap and NodeQueue do not, so copying them could lead to memory-management problems such as double deletion. Gemini also has more code for locating nodes through level-order traversal. Claude's code is more straightforward to maintain, while Gemini's pointer-based structure is more complex. Both could be improved with more complete input validation, especially for dates and malformed email commands.

f. Select one to use as the basis for this assignment and justify why you picked it.
I would choose Claude's implementation because it has O(logn) insertion and removal, with amortized O(logn) insertion, making it more efficient for larger inboxes. Its dynamically resized array is also easier to manage, and I like that it uses arrival order to handle emails with the same category and date. Gemini's code correctly uses a pointer-based MaxHeap, but finding the insertion parent and last node requires a level-order traversal, making insertion and removal O(n). The assignment asks for a "list-based" MaxHeap, which I interpreted as an array-based list. If it means a linked, pointer-based structure, Gemini's approach may fit that requirement better. However, based on my interpretation of the requirement and the efficiency of the heap operations, I would choose Claude's implementation.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I improved the program by adding input validation for sender categories, dates, subjects, and missing fields. Invalid emails are skipped, and the program displays a warning in stderr explaining the problem. I also added a warning for unknown commands and made READ display a message when the inbox is empty, just like NEXT already did. The original code used stoi to convert the date, which could cause the program to crash if the date contained invalid characters. The improved code parses the date manually and checks its format and basic ranges before adding the email. I kept the original arrival-order tie-breaker so emails with the same category and date are still handled consistently.

Execution Time: There was no major change to the execution time. Insertion and removal remain O(logn), with insertion being amortized O(logn) because the array occasionally needs to grow. Checking the next email and counting emails remain O(1). The new input validation adds only a constant amount of work for each email, so it does not change the overall time complexity.

Space Complexity: The space complexity remains O(n) because the program still stores emails in a dynamically resized array. The main change is using unique_ptr to manage the array automatically. This does not reduce the amount of space used, but it makes memory cleanup safer and removes the need to manually delete the array.

Maintainability: I renamed the classes to Message, EmailHeap, and InboxManager so the program is my own revised version. I also added separate helper methods for checking sender categories and validating and converting dates, which keeps the input-validation code organized. Using unique_ptr removes the need for a hand-written destructor to delete the array. I added comments throughout the code to explain what each part does. Finally, the program can now read commands from a file passed as a command-line argument or from standard input, without prompting the user for a filename.